



**UNIVERSIDAD
NACIONAL DE
INGENIERÍA**



TRANSFORMACIÓN
digital

CURSOS GRATUITOS

OTI UNI

Git y GitHub desde cero

Sesión 01: Fundamentos, instalación y trabajo local

Instructor:  **Anthony Gonzalo Quispe Cordova**

Oficina de Tecnologías de la Información (OTI)

19 de mayo de 2026

Flujo de Cursos Orientado a DevOps



Horarios de los Cursos del Flujo DevOps



Fundamentos de Linux



Sábados



9:00 a.m. – 12:00 p.m.



23/05 – 27/06 | 18 h



Git y GitHub



Lun, Mar, Jue, Vie



4:00 – 7:00 p.m.



19/05 – 28/05 | 18 h



Fundamentos de Docker



Lun, Mar, Vie



4:00 – 7:00 p.m.



29/05 – 09/06 | 18 h



Ansible: Automatización



Sábados



2:00 – 5:00 p.m.



23/05 – 27/06 | 18 h

Índice de Contenidos

- 1** Introducción
- 2** Un poco de teoría
- 3** Un poco de Linux
- 4** Instalando y Configurando Git
- 5** Trabajando con Git de forma local
- 6** Cierre

Introducción

Bienvenida y contexto

- El objetivo de esta sesión es entender qué problema resuelve Git antes de memorizar comandos.
- Trabajaremos con ejemplos locales para construir una base sólida antes de pasar a colaboración remota.
- Al final de la sesión tendrás tu primer repositorio funcionando desde cero.

Requisitos previos

Conocimientos

- Manejo básico de archivos y carpetas.
- Nociones simples de terminal.
- Ganas de practicar, equivocarse y corregir.

Herramientas

- Git instalado o listo para instalar.
- Editor de código.
- Terminal: Git Bash, PowerShell, CMD, macOS Terminal o Linux Shell.

Un poco de teoría

¿Qué es un control de versiones?

Un control de versiones guarda la historia de un proyecto para saber **qué cambió, cuándo y por qué.**

🕒 Historial

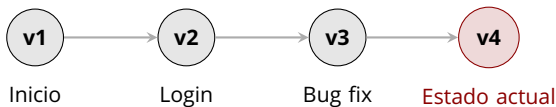
Registra cambios importantes en una línea de tiempo.

🔄 Recuperación

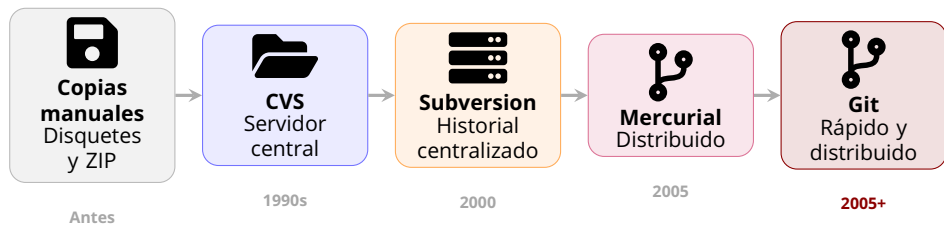
Permite volver a un punto anterior si algo falla.

🔍 Trazabilidad

Ayuda a responder quién cambió qué y cuándo.



Evolución de los sistemas de control de versiones



Idea central

La evolución fue desde copiar archivos manualmente hacia sistemas que guardan historial, facilitan colaboración y permiten trabajar sin depender siempre de un servidor central.

¿Por qué es importante?



Seguridad

Recuperas versiones anteriores sin depender de copias manuales.



Orden

Cada cambio queda documentado en una línea de tiempo clara.



Colaboración

Varias personas trabajan sobre el mismo proyecto con reglas claras.

¿Qué es Git?

- Git es un sistema de control de versiones **distribuido**.
- Cada repositorio local contiene el historial del proyecto.
- Fue creado por **Linus Torvalds** para gestionar proyectos grandes y cambiantes.
- No necesitas internet para guardar cambios localmente con `commit`.



Linus Torvalds

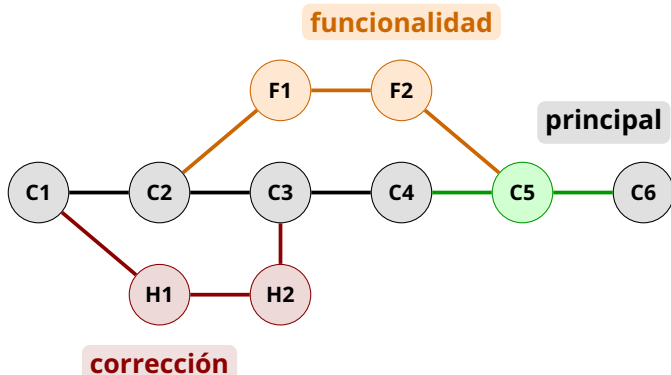
Vista previa: ¿qué es una rama?

- Una rama es una línea de trabajo dentro del historial del proyecto.
- Permite separar cambios: una idea, una corrección o una nueva funcionalidad.
- En esta sesión solo veremos la idea; el trabajo práctico con ramas vendrá después.

Ejemplo mental

La rama principal conserva la versión estable; una rama secundaria permite experimentar sin afectar esa línea estable.

Visualizando ramas en Git



Las ramas ayudan a trabajar cambios separados y luego integrarlos al historial principal.

master y main: nombres de la rama principal

- Ambas suelen representar la rama principal del proyecto.
- Algunos repositorios antiguos usan `master`; muchos proyectos nuevos usan `main`.
- En esta sesión solo necesitamos reconocer el nombre que Git muestre al inicializar el repositorio.
- El manejo práctico de ramas se trabajará más adelante.

¿Git y GitHub son lo mismo?



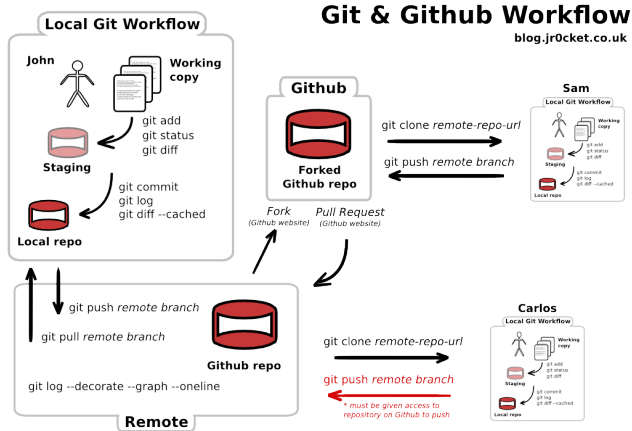
Git

- Herramienta instalada en tu equipo.
- Controla versiones localmente.
- Funciona sin internet.

GitHub

- Plataforma web para alojar repositorios.
- Facilita colaboración, revisión y publicación.
- Usa Git como base.

Relación entre Git y GitHub



Un poco de Linux

Linux y la terminal: base del trabajo moderno

- Linux está presente en servidores, nube, contenedores y automatización.
- La terminal permite trabajar de forma rápida, repetible y precisa.
- Git se usa mucho desde línea de comandos porque muestra claramente el estado del proyecto.
- Dominar lo básico facilita avanzar hacia Docker, Ansible y CI/CD.

Idea clave

No necesitamos ser expertos en Linux para usar Git, pero sí movernos con confianza en la terminal.



Linux + Terminal

Base práctica para Git y DevOps

Manos a la obra

Practica comandos básicos de Linux directamente en el navegador:

<https://kodekloud.com/studio/labs/linux/>

Familiarizarse con la terminal ayuda a entender mejor los comandos de Git.



Guía rápida oficial de comandos Ubuntu

Usa este cheatsheet como apoyo para practicar navegación, archivos, paquetes, usuarios y red en la terminal.



Descargar / abrir PDF

https://assets.ubuntu.com/v1/d00791ae-ubuntu_cli_cheat_sheet_2025.pdf

Instalando y Configurando Git

¿Cómo saber si tengo Git instalado?

Comando de verificación

```
git --version
```

- Si aparece una versión, Git está instalado.
- Si el sistema no reconoce el comando, necesitas instalarlo o revisar el PATH.

¿Cómo instalar Git?



Windows

```
git-scm.com
```

Descarga el instalador
y activa **Git Bash**.



macOS

```
brew install git
```

Alternativa: Xcode
Command Line Tools.



Linux

```
sudo apt  
install git
```

Usa el gestor de
paquetes de
tu distribución.

📄 [Página general de descargas: git-scm.com/downloads](https://git-scm.com/downloads)

Después de instalar

Verifica la instalación con `git --version`.

Configurar nombre y correo en Git

Identidad global

```
git config --global user.name "Tu Nombre"  
git config --global user.email "tu@email.com"
```

- Git usa estos datos para firmar cada commit.
- Usa el correo que quieras asociar a tus proyectos.
- La opción `-global` aplica la configuración a todos tus repositorios del usuario actual.

Configurar el editor por defecto

Ejemplo con Visual Studio Code

```
git config --global core.editor "code --wait"
```

- El editor se abre cuando Git necesita que escribas mensajes o resuelvas operaciones.
- Puedes usar otro editor si es el que tienes instalado y dominas.

Comprobar configuración de Git

Ver toda la configuración

```
git config --list
```

Consultar un valor específico

```
git config user.name  
git config user.email
```

Trabajando con Git de forma local

¿Cómo inicializar un nuevo proyecto Git?

Crear un repositorio local

```
mkdir mi-proyecto  
cd mi-proyecto  
git init
```

- `git init` crea la carpeta oculta `.git`.
- Desde ese momento Git puede registrar el historial del proyecto.

¿Qué es el directorio de trabajo?

- Es la carpeta visible donde editas, creas y borras archivos.
- Git compara ese estado contra el último commit conocido.
- Cuando un archivo cambia, Git puede verlo como modificado o sin seguimiento.

Los tres estados en Git



Idea clave

Primero modificas, luego preparas lo que quieres guardar y finalmente confirmas el cambio.

Mapa del flujo local

Los 3 Estados de Git

El ciclo básico de los archivos en Git



Working Directory
(modified)

Aquí editas tus archivos.
Los cambios aún no están
preparados.



`git add`



Staging Area
(staged)

Los cambios están preparados
y listos para confirmar.



`git commit`



Local Repository
(committed)

Los cambios están guardados
de forma permanente en tu
repositorio local.

Revisar el estado con `git status`

Comando esencial

```
git status
```

- Muestra archivos modificados, preparados y sin seguimiento.
- También indica si el repositorio está limpio o si hay cambios pendientes.

Añadir archivos al área de staging

Preparar cambios

```
git add archivo.txt  
git add .
```

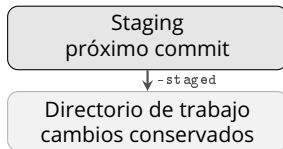
- `git add archivo.txt` prepara un archivo específico.
- `git add .` prepara todos los cambios del directorio actual.
- Staging permite elegir exactamente qué entrará en el próximo commit.

Sacar archivos del área de staging

Quitar del próximo commit sin borrar el trabajo

```
git restore --staged archivo.txt
```

- Saca el archivo del área de staging.
- Tus cambios **siguen existiendo** en el directorio de trabajo.
- Úsalo si hiciste `git add` por error o quieres separar commits.

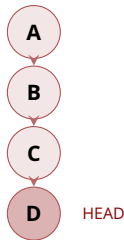


¿Qué es un commit?

- 📷 Instantánea confirmada del estado del proyecto.
- 🔑 Tiene un identificador único (SHA).
- 👤 Registra autor, fecha y mensaje.
- 📍 Es un punto al que puedes volver.

Regla práctica

Si no puedes resumir el cambio en una frase clara, el commit es demasiado grande.



Haz commits atómicos

- ⚙️ Un commit = un solo cambio lógico.
- 👁️ Facilita revisar y entender el historial.
- ↶ Permite revertir cambios con precisión.
- 🚫 Evita mezclar tareas distintas en un mismo commit.

✓ Atómico

Agrega validación del formulario de registro

✗ No atómico

Agrega login, corrige CSS y actualiza readme

¿Cómo hacer un commit?

Flujo completo

```
git add archivo.txt  
git status  
git commit -m "Describe el cambio"
```

- ✓ Revisa el estado antes de confirmar.
- ✎ El mensaje debe ser claro y en modo imperativo.

👍 Buenos mensajes

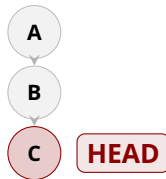
Agrega botón de login
Corrige validación de email
Actualiza dependencias

👎 Evita

cambios, fix, asdf, wip

¿Qué es HEAD?

- 📌 HEAD es un puntero a tu posición actual.
- 🔗 Normalmente apunta al último commit de la rama activa.
- ⚙️ Muchos comandos usan HEAD para comparar o restaurar.

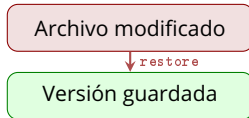


¿Cómo deshacer un archivo modificado?

Descartar cambios del directorio de trabajo

```
git restore archivo.txt
```

- Reemplaza tu archivo con la última versión guardada por Git.
- Sirve cuando probaste algo y quieres volver atrás.
- Solo funciona si el archivo ya había sido confirmado antes; si es nuevo, Git no tiene una versión previa a la que regresar.



Cuidado

Pierdes los cambios que no hayan sido guardados previamente con un commit.

¿Cómo puedo deshacer mis cambios?

1. Solo modificado

```
git restore archivo.txt
```

Descarta cambios locales del archivo.

2. En staging

```
git restore -staged  
archivo.txt
```

Lo quita del próximo commit, pero conserva el trabajo.

3. Ya hice commit

Usa `git revert` o crea un commit correctivo. Evita borrar historial compartido.

Regla de seguridad

Antes de deshacer, ejecuta `git status`. Si el cambio no está en un commit, `git restore archivo.txt` puede perderlo.

Ignorar archivos con .gitignore

- .gitignore indica qué archivos no debe rastrear Git.
- Se usa para dependencias, archivos temporales, binarios y secretos locales.

Ejemplo

```
node_modules/  
*.log  
.env
```

Ignorar siempre los mismos archivos

- Algunos archivos dependen de tu sistema o editor y no deberían repetirse en cada proyecto.
- Para eso puedes usar un `.gitignore` global.

Casos comunes

Archivos del sistema operativo, configuraciones personales del editor y temporales locales.

Comandos básicos en el ciclo de vida

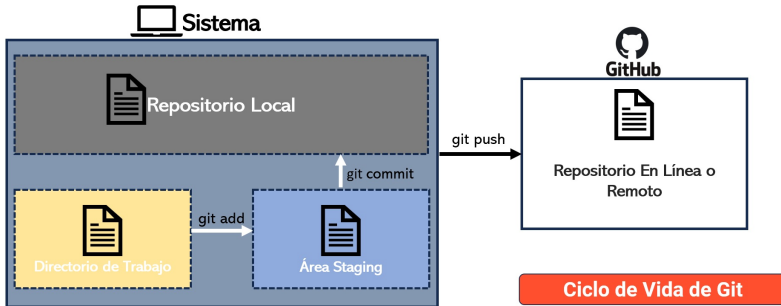
Máster en Git y GitHub Moderno



git



GitHub



Laboratorio: primer flujo local

- 1 Crea una carpeta e inicializa Git con **git init**.
- 2 Crea **README.md** y revisa el estado con **git status**.
- 3 Prepara el archivo con **git add README.md**.
- 4 Crea un commit con **git commit -m 'Agrega README inicial'**.
- 5 Modifica el archivo, deshaz el cambio y vuelve a revisar el estado.
- 6 Crea **.gitignore** e ignora archivos ***.tmp**.

Meta

Recorrer el ciclo básico: modificar → preparar → confirmar → revisar.

Resumen de la sesión

-  Git: control de versiones distribuido.
-  Flujo: modificar  preparar  confirmar.
-  La config define tu identidad global.
-  `git status`: tu brújula principal.
-  Commits atómicos = historial legible.
-  `.gitignore`: mantén limpio el repo.

Próxima sesión

Historial y control avanzado: `git diff`, `git log` detallado y cómo recuperar errores.

Cierre